

Automatically Generate operator->

Document Number: P3039R0

Date: 2023-11-07

Author: David Stone (davidfromonline@gmail.com)

Audience: Evolution Working Group (EWG), Library Evolution Working Group (LEWG)

Summary

This proposal follows the lead of `operator<=>` (see [Consistent Comparison](#)) by generating rewrite rules if a particular operator does not exist in current code. This is a follow-up to P1046R2, but it includes only `operator->` and `operator->*`.

- Rewrite `lhs->rhs` as `(*lhs).rhs`
- Rewrite `lhs->*rhs` as `(*lhs).*rhs`

Design Goals

The primary goal of this paper is that users should have little or no reason to write their own version of an operator that this paper proposes generating. It would be a strong indictment if there are many examples of well-written code for which this paper provides no simplification, so a fair amount of time in this paper will be spent looking into the edge cases and making sure that we generate the right thing. At the very least, types that would not have the correct operator generated should not generate an operator at all. In other words, it should be uncommon for users to define their own versions (the default should be good enough most of the time), and it should be extremely rare that users want to suppress the generated version (= `delete` should almost never appear).

This paper was split off from P1046R2. That paper proposed generating many operators, this paper is just the arrow overloads. This is the part of that paper with the most motivation (as users cannot write the equivalent), the feature that gained the most support from the previous EWG meeting, and one of the least complicated changes.

This area has been well-studied for library solutions. This paper, however, traffics in rewrite rules (following the lead of `operator<=>`), not in terms of function calls. Because of this, we have one more option that the library-only solutions lack: we could define `lhs->rhs` as being equivalent to `(*lhs).rhs`. This neatly sidesteps all of the issues of library-only solutions (how do we get the address of the object? how do we handle temporaries?). It even plays nicely with existing rules around lifetime extension of temporaries. This solves many long-standing issues around proxy iterators that return by value.

Comparison with comparison rewrites

This change tries to follow the lead of the existing rewrite rules we have. However, unlike the comparison operators this change is significantly simpler and less likely to have all the corner cases we've seen with `operator==` and `operator<=>`. Much of the complexity around the comparison operators has come from reversed candidates and dealing with potentially multiple arguments. The specification of these operators should be significantly simpler.

operator->

`operator->` is the simplest. It cannot be defined outside of the class, so there is exactly one place to look for whether it exists today. It is also a unary operator (it does not depend on the right-hand side). This makes

overload resolution and name lookup very simple.

operator->*

operator->* is slightly more complicated. It can be defined as a free operator, a member function, and a hidden friend. However, we've already solved the problems associated with that for comparison operators. It is a binary operator, but the order is meaningful and thus there are no complexities around reversed candidates. One of the base operators for this, operator.*, also cannot be overloaded. This means that the only overloaded operator that is relevant to operator->* is the same as for operator->: operator*, which is a unary operator. That fact also eliminates much of the complexity of operator== vs operator!=.

Design

If any of this conflicts with how operator!= is defined in terms of operator== and is not explicitly called out as a difference, that difference is unintentional.

All of these examples assume there is a variable lhs of type LHS.

operator->

If the expression lhs->rhs is encountered:

- If overload resolution for operator-> would succeed, then that operator is called. This can also select a deleted overload.
- If overload resolution does not succeed, but the expression *lhs is well-formed, then the expression is rewritten to (*lhs).rhs
- Otherwise, the expression is ill-formed

operator->*

If the expression lhs->*rhs is encountered:

- If overload resolution for operator->* would succeed, then that operator is called. This can also select a deleted overload.
- If overload resolution does not succeed, but the expression *lhs is well-formed, then the expression is rewritten to (*lhs).*rhs
- Otherwise, the expression is ill-formed

Library impact

Given the path we took for operator<=> of removing manual definitions of operators that can be synthesized and assuming we want to continue with that path by approving ["Do not promise support for function syntax of operators"](#), this will allow removing a large amount of specification in the standard library by replacing it with blanket wording that these rewrites apply. I have surveyed the standard library to get an overview of what would change in response to this, and to ensure that the changes would work properly. This covers every type that was in the standard library as of early 2020.

Types that will gain operator-> and this is a good thing

- move_iterator currently has a deprecated operator->
- counted_iterator
- istreambuf_iterator

- `istreambuf_iterator::proxy` (exposition only type)
- `iota_view::iterator`
- `transform_view::iterator`
- `split_view::outer_iterator`
- `split_view::inner_iterator`
- `basic_istream_view::iterator`
- `elements_view::iterator`

Most of these are iterators that return either by value or by `decltype(auto)` from some user-defined function. It is not possible to safely and consistently define `operator->` for these types, so we do not always do so, but under this proposal they would all do the right thing.

Types that will technically gain `operator->` but it is not observable

- `insert_iterator`
- `back_insert_iterator`
- `front_insert_iterator`
- `ostream_iterator`

The insert iterators and `ostream_iterator` technically gain an `operator->`, but `operator*` returns a reference to `*this` and the only members of those types are types, constructors, and operators, none of which are accessible through `operator->` using the syntaxes that are supported to access the standard library.

Types that will gain `operator->` and it's weird either way

- `ostreambuf_iterator`

`ostreambuf_iterator` is the one example for which we might possibly want to explicitly delete `operator->`. It has an `operator*` that returns `*this`, and it has a member function `failed()`, so it would allow calling `it->failed()` with the same meaning as `it.failed()`.

Types that have `operator->` now and it behaves the same as the synthesized operator

All types in this section have an `operator->` that is identical to the synthesized version if we do not wish to support users calling with the syntax `thing.operator->()`.

- `optional`
- `unique_ptr` (single object)
- `shared_ptr`
- `weak_ptr`
- `basic_string::iterator`
- `basic_string_view::iterator`
- `array::iterator`
- `deque::iterator`
- `forward_list::iterator`
- `list::iterator`
- `vector::iterator`
- `map::iterator`
- `multimap::iterator`
- `set::iterator`
- `multiset::iterator`
- `unordered_map::iterator`
- `unordered_set::iterator`
- `unordered_multimap::iterator`

- `unordered_multiset::iterator`
- `span::iterator`
- `istream_iterator`
- `valarray::iterator`
- `tzdb_list::const_iterator`
- `filesystem::path::iterator`
- `directory_iterator`
- `recursive_directory_iterator`
- `match_results::iterator`
- `regex_iterator`
- `regex_token_iterator`
- `reverse_iterator`
- `common_iterator`
- `filter_view::iterator`
- `join_view::iterator`

All of these types that are adapter types define their `operator->` as deferring to the base iterator's `operator->`. However, the Cpp17InputIterator requirements specify that `a->m` is equivalent to `(*a).m`, so anything a user passes to `reverse_iterator` must already meet this. `common_iterator`, `filter_view::iterator`, and `join_view::iterator` were added in C++20 and require `input_or_output_iterator` of their parameter, which says nothing about `->`. Its `operator->` is defined as the first in a series that compiles:

- 1) Try calling member `operator->` on the base iterator
- 2) Try taking the address of the value returned from `operator*`
- 3) Create a proxy object that stores by-value returns and returns the address of that

If this paper were accepted, we have two options.

- 1) Get rid of the manual definition of `operator->` from those new types, which is a breaking change for iterator types with an `operator->` that does something meaningfully different from what their `operator*` does, or
- 2) Manually define it only when the wrapped type has a member `operator->`. This would keep step 1, but eliminate steps 2 and 3.

iterator_traits

`std::iterator_traits<I>::pointer` is essentially defined as `typename I::pointer` if such a type exists, otherwise `decltype(std::declval<I &>().operator->())` if that expression is well-formed, otherwise `void`. The type appears to be unspecified for iterators into any standard container, depending on how you read the requirements. The only relevant requirement on standard container iterators (anything that meets Cpp17InputIterator) are that `a->m` is equivalent to `(*a).m`. We never specify that any other form is supported, nor do we specify that any of them contain the member type `pointer`. There are three options here:

1. Change nothing. This would make `pointer` defined as `void` for types that have a synthesized `operator->`
2. Specify a further fallback of `decltype(std::addressof(*a))` to maintain current behavior and allow users to delete their own `operator->` without changing the results of `iterator_traits`
3. Deprecate or remove the `pointer` typedef, as it is not used anywhere in the standard except to define other `pointer` typedefs and it seems to have very little usefulness outside the standard.

My recommendation is 2, 3, or both.

to_address and pointer_traits

[`pointer.conversion`] specifies `to_address` in terms of calling `p.operator->()`, so some thought will need to be put in there on what to do.

The following standard types can be used to instantiate `pointer_traits`:

- `T *`
- `unique_ptr`
- `shared_ptr`
- `weak_ptr`
- `span`

However, none of them are specified to have member `to_address`.

Note that `span` does not have `operator->` and is thus not relevant to the below discussion at all. `unique_ptr`, `shared_ptr`, and `weak_ptr` are not iterators, and are thus minimally relevant to the below discussion.

`std::to_address` is specified as calling `pointer_traits<Ptr>::to_address(p)` if that is well formed, otherwise calling `operator->` with member function syntax. This leaves us with several options:

1. Leave this function as-is and specify that all of the types that currently have `operator->` have a specialization of `pointer_traits` that defines `pointer_traits<T>::to_address`
2. Specify that all types that currently have `operator->` work with `std::to_address`
3. Define a second fallback if `p.operator->()` is not valid that would be defined as `std::addressof(*p)`.
This is similar to the question for `std::iterator_traits<I>::pointer`.

1 and 2 feel like the wrong approach -- they would mean that authors of iterator types still need to define their own `operator->`, or they must specialize some class template (if we agree that the current semantics with regard to iterators are correct), or they must overload `to_address` and we make that a customization point found by ADL.